



Informe técnico

Procesamiento paralelo de imágenes
con Scheme y Erlang

Lenguajes de Programación

Estudiante:

Daniel Araya Pérez

Carné: 2025078995

Profesor:

Eddy Ramírez Jiménez

Setiembre de 2026

1. Resumen

El proyecto combina Erlang para coordinar procesos y Scheme para transformar imágenes PPM P3. Erlang divide la imagen, ejecuta una instancia de Scheme por región y reúne los resultados. Se implementaron difuminado gaussiano y escala de grises. El rendimiento se midió con 1, 2, 4 y 8 procesos.

2. Diseño de la solución

2.1. Arquitectura

El archivo `image_processor` inicia el programa. En Erlang, `main.erl` recibe los parámetros, `ppm.erl` lee la imagen, `partition.erl` crea las regiones y `coordinator.erl` dirige la ejecución. Cada proceso de `worker.erl` usa `scheme_bridge.erl` para ejecutar su propia instancia de `scheme/worker.scm`. Al terminar, el coordinador ordena las regiones y escribe la imagen completa.

Una imagen se representa mediante su ancho, su alto y una lista de filas. Cada píxel tiene tres componentes: rojo, verde y azul. Los lectores y escritores de Erlang y Scheme usan PPM P3, la variante de texto del formato [3], con valor máximo 255. Los módulos Scheme usan Full Swindle [2].

2.2. División de la imagen

La imagen se divide en franjas horizontales de ancho completo. Para una altura H y p procesos, cada franja recibe al menos $\lfloor H/p \rfloor$ filas. Las $H \bmod p$ filas restantes se reparten, una por una, entre las primeras franjas. Por eso, la cantidad de procesos no puede superar la altura.

El difuminado necesita filas vecinas de otras franjas. Para un kernel de tamaño impar k , cada región recibe hasta $r = (k-1)/2$ filas de contexto por arriba y por abajo. Estas filas forman el halo. El proceso devuelve solamente sus filas asignadas. La escala de grises no necesita halo. Como las franjas ocupan todo el ancho, no se agregan columnas de contexto.

2.3. Concurrencia y reconstrucción

El coordinador crea un proceso Erlang por región con `spawn_monitor` [1]. Cada proceso envía un mensaje `region_completed` al terminar. La recepción ocurre conforme llegan los mensajes, sin exigir que los procesos terminen en el orden de las franjas. Después, Erlang ordena las regiones por su fila inicial y concatena sus filas para reconstruir la imagen.

3. Comunicación entre Erlang y Scheme

El intercambio usa archivos temporales y argumentos de línea de comandos. Para cada región, Erlang crea una carpeta única en `/tmp` con tres rutas: entrada, salida y estado. Escribe la región, incluido su halo, como PPM P3 y ejecuta `scheme/worker.scm` mediante el comando `racket`.

Se eligieron archivos para reutilizar los lectores y escritores PPM de ambos lenguajes sin gestionar conexiones. Las carpetas separadas permiten ejecutar trabajadores en paralelo sin compartir rutas, a cambio de más operaciones de disco.

Los argumentos indican las tres rutas, el filtro y los límites de las filas que deben devolverse, expresados respecto de la región con halo. Para `gaussian` también incluyen el tamaño del kernel y sigma. Scheme lee la entrada, aplica el filtro, escribe solo la parte asignada y al final escribe `ok` en el archivo de estado.

Erlang acepta el resultado solo si encuentra ese estado. Después lee la salida, comprueba sus dimensiones y elimina los archivos temporales del trabajador. Cada proceso Erlang espera a su propia instancia de Scheme, pero los distintos procesos Erlang trabajan de forma concurrente.

4. Filtros y bordes

4.1. Difuminado gaussiano

`gaussian_blur.scm` construye el kernel a partir de su tamaño y sigma. El peso de cada posición (x, y) es proporcional a

$$w(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \quad (1)$$

Los pesos se dividen entre su suma para que el total sea uno [4]. `convolution.scm` combina cada componente RGB con esos pesos, redondea el resultado y lo limita al intervalo de 0 a 255. El tamaño predeterminado es 3×3 y el sigma predeterminado es 0,8493. También se aceptan tamaños impares desde 3 y valores positivos de sigma.

Cuando una posición del kernel queda fuera de la imagen, se usa el píxel más cercano del borde. Scheme aplica esta regla tanto en filas como en columnas. El halo proporciona los píxeles reales de las franjas vecinas, de modo que un límite entre procesos no se trata como borde de la imagen.

4.2. Escala de grises

`grayscale.scm` calcula

$$Y = \text{round}(0,299R + 0,587G + 0,114B) \quad (2)$$

y produce el píxel (Y, Y, Y) . MathWorks presenta estos coeficientes en su documentación sobre conversión de RGB a gris [5]. Este filtro no usa kernel ni sigma.

5. Manejo de errores

Erlang rechaza parámetros inválidos, como una cantidad de procesos no positiva, un kernel par o rutas de entrada y salida iguales. Si Scheme falla antes de escribir `ok`, Erlang detecta que falta el estado de éxito. El trabajador termina con error y el monitor informa al coordinador.

Ante el fallo de un trabajador, el coordinador cancela los pendientes e informa el problema. La imagen final solo se escribe cuando llegaron todas las regiones válidas, por lo que un fallo no genera una imagen incompleta. No se implementó la repetición automática de una región fallida.

6. Evaluación del rendimiento

6.1. Método

Se usó `samples/sample_1.ppm`, de 640×413 píxeles, con el difuminado predeterminado: kernel 3×3 y sigma 0,8493. Las mediciones se hicieron en Ubuntu bajo WSL, con 12 procesadores lógicos, Racket 9.3 y el emulador Erlang 13.2.2.5. Se realizaron ocho corridas con 1 y 2 procesos y dieciséis con 4 y 8 procesos. La tendencia general se repitió entre corridas. El tiempo informado por el programa incluye lectura, división, procesamiento, reconstrucción y escritura de la imagen. No incluye la compilación previa del lanzador ni el arranque de la máquina virtual Erlang.

Se toma la mediana de las mediciones como T_p para reducir el efecto de valores aislados. La aceleración y la eficiencia se calculan mediante

$$S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}. \quad (3)$$

Procesos	Corridas	T_p (s)	S_p	E_p (%)
1	8	4.228	1.00	100.0
2	8	2.234	1.89	94.6
4	16	1.682	2.51	62.8
8	16	1.504	2.81	35.1

Cuadro 1: Medianas de tiempo, aceleración y eficiencia

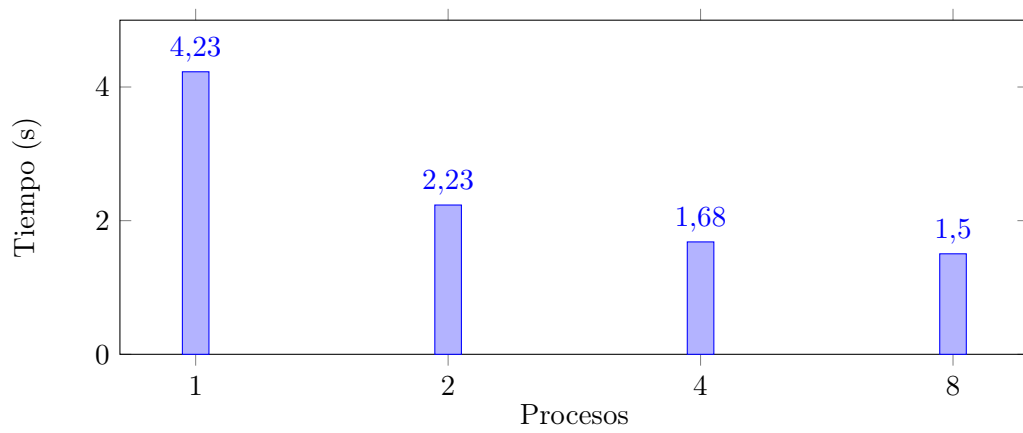


Figura 1: Tiempo mediano en segundos por cantidad de procesos

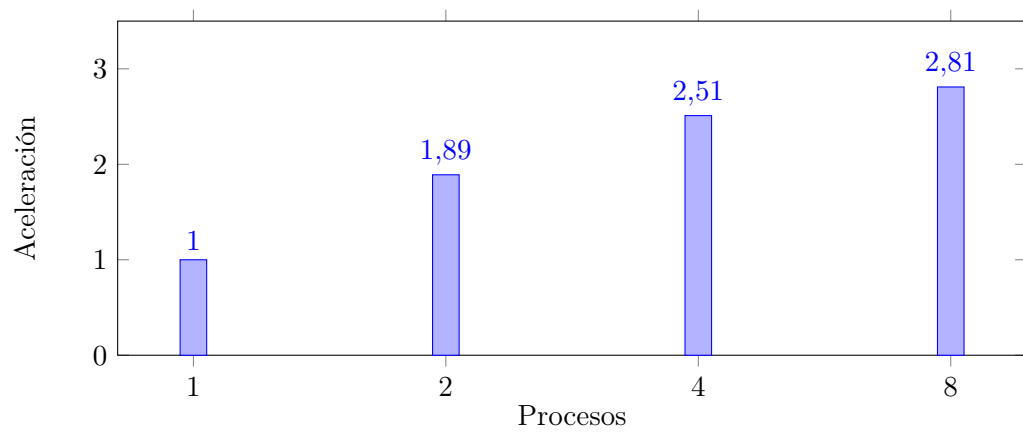


Figura 2: Aceleración por cantidad de procesos

6.2. Análisis

La mediana baja de 4.228 segundos con un proceso a 1.504 segundos con ocho. La mayor aceleración medida es 2,81, con ocho procesos. La mejora de cuatro a ocho procesos es menor que la de uno a dos. Algunas corridas fueron más lentas, pero la tendencia general se mantuvo.

El tiempo no se dedica solo al filtro. Cada región requiere crear un proceso Scheme, escribir y leer archivos PPM, y copiar datos entre Erlang y Scheme. También se pagan la división y la reconstrucción. Estos costos reducen la eficiencia al aumentar p . No se midió cada etapa por separado.

7. Limitaciones

- Solo se aceptan imágenes PPM P3 válidas, con valor máximo 255 y sin comentarios.
- Cada región inicia una instancia nueva de Scheme y usa archivos temporales, lo que agrega trabajo a cada ejecución.
- Un trabajador fallido se informa, pero su región no se vuelve a procesar automáticamente.
- Si se cancela un trabajador mientras usa su carpeta temporal, pueden quedar archivos en `/tmp`.
- Las rutas se colocan entre comillas simples en un comando de shell. Una ruta que contenga una comilla simple puede fallar.

8. Conclusiones

Erlang resulta adecuado para coordinar las regiones porque permite crear procesos concurrentes, recibir sus resultados por mensajes y detectar fallos con monitores. Scheme resulta adecuado para los filtros porque sus funciones recursivas procesan listas de filas y píxeles, y construyen el kernel sin modificar la imagen de entrada.

Los halos conservan los vecinos necesarios en los límites entre regiones. En la imagen medida, ocho procesos dieron el menor tiempo, pero la eficiencia bajó al aumentar la cantidad de procesos por el trabajo adicional de coordinación y comunicación.

Referencias

- [1] Erlang, *Processes*, Erlang System Documentation. https://www.erlang.org/doc/system/ref_man_processes.html.
- [2] Racket, *Swindle*, Racket Documentation. <https://docs.racket-lang.org/swindle/index.html>.
- [3] Netpbm, *PPM Format Specification*. <https://netpbm.sourceforge.net/doc/ppm.html>.
- [4] A. Torralba, P. Isola y W. Freeman, *Foundations of Computer Vision*, sección 17.3, Gaussian Filter. https://visionbook.mit.edu/blurring_2.html.
- [5] MathWorks, *rgb2gray: Convert RGB Image or Colormap to Grayscale*, sección Algorithms. <https://www.mathworks.com/help/matlab/ref/rgb2gray.html>.